포인터와 mm 함수 통합 학습 가이드

이 문서는 재혁님이 오늘 포인터 감각을 확실히 잡고, 이번 주 `malloc-lab`의 `mm` 함수들과 수요 코딩회의 트리/노드 포인터까지 한 흐름으로 이해할 수 있게 만든 학습용 가이드입니다. 목표는 **코드를 외우는 것보다 "포인터가 무엇을 가리키는지"를 정확히 보는 눈**을 만드는 것입니다.

pointer · payload/header/footer · implicit free list · mm_init · mm_malloc · mm_free · mm_realloc · extend_heap · find_fit · place · coalesce · B+ tree

이 문서를 보는 방법

- 코드 문법을 외우기보다, 각 포인터가 **어느 위치를 가리키는지** 먼저 보세요.
- `payload` 포인터, `header` 포인터, `footer` 포인터, `next block` 포인터를 구분해서 읽으세요.
- 오늘은 구현 완성보다 **왜 이런 포인터 계산이 필요한지**를 이해하면 충분합니다.

1. 오늘 가장 먼저 잡아야 하는 한 문장

`malloc`이 사용자에게 돌려주는 것은 payload의 시작 주소이고, allocator는 그 앞뒤에 숨겨진 header/footer/padding까지 포함한 **block 전체**를 관리합니다.

이 한 문장을 정확히 이해하면 재혁님이 오늘까지 헷갈렸던 대부분이 풀립니다. 포인터는 단순히 "주소를 담는 변수"가 아니라, 지금 상황에 따라 **payload를 가리키기도 하고, 그 payload의 앞쪽 header를 찾기 위한 기준점**이 되기도 합니다.

2. 포인터를 이번 주 관점으로 다시 정의하기

일반 C 포인터

포인터는 어떤 메모리 위치의 주소를 저장하는 변수입니다. `int *`는 정수 주소, `char *`는 바이트 단위 이동이 쉬운 주소, `void *`는 타입 없는 일반 주소 느낌으로 자주 씁니다.

이번 주 포인터

이번 주에는 포인터를 **데이터를 찾기 위한 주소**라기보다 **블록 구조를 읽고 힙을 순회하기 위한 기준점**으로 자주 보게 됩니다.

이름	무엇을 가리키나	왜 중요한가
payload pointer	사용자가 실제 데이터를 쓰는 시작 주소	`malloc`이 반환하는 주소이기 때문
header pointer	현재 block의 크기/상태 정보가 있는 위치	size와 alloc bit를 읽어야 다음 block을 계산할 수 있음
footer pointer	boundary tag 방식에서 block 끝의 복사 정보 위치	이전 block을 빠르게 찾아 coalesce하기 위해 사용
next block pointer	현재 block 크기만큼 이동한 다음 block 위치	implicit free list를 순회하는 핵심

3. 블록 구조를 주소로 보는 연습

아래 예시는 `malloc(15)` 요청을 처리할 때 그림 9.35류의 단순 block 형식을 주소 감각으로 적어본 것입니다.

예시 1: header + payload + padding

```
주소 100 ~ 103 : header 4바이트
주소 104 ~ 118 : payload 15바이트
주소 119 ~ 123 : padding 5바이트
총 블록 크기 : 24바이트
```

- 1줄: allocator는 block 맨 앞에 관리 정보용 header를 둡니다.
- 2줄: 사용자가 실제로 쓰는 공간은 payload입니다.
- 3줄: 8바이트 정렬을 맞추기 위해 남는 칸은 padding으로 둡니다.
- 4줄: `15 + 4 = 19`를 8의 배수로 올리면 24가 됩니다.

여기서 꼭 구분하기

- **사용자 관점**: 나는 15바이트를 쓸 수 있다.
- **allocator 관점**: 나는 24바이트 block 하나를 관리한다.
- **반환 주소**: header 주소가 아니라 payload 시작 주소다.

4. starter mm.c를 포인터 관점으로 해석하기

현재 재혁님 폴더의 [`mm.c`](/C:/jungle3/malloc-lab/mm.c)는 아주 단순한 starter 코드입니다. reuse도 없고 coalesce도 없지만, 오히려 처음 포인터 감각을 잡기에는 좋습니다.

예시 2: starter mm_malloc 핵심 부분

```
1  void *mm_malloc(size_t size)
2  {
3      int newsize = ALIGN(size + SIZE_T_SIZE);
4      void *p = mem_sbrk(newsize);
5      if (p == (void *)-1)
6          return NULL;
7      else {
8          *(size_t *)p = size;
9          return (void *)((char *)p + SIZE_T_SIZE);
10     }
11 }
```

- 1줄: `size`바이트를 달라는 요청을 받습니다.
- 3줄: payload 크기에 관리 정보 크기(`SIZE_T_SIZE`)를 더하고 정렬합니다.
- 4줄: `mem_sbrk`로 힙을 그만큼 늘리고 시작 주소를 받습니다.
- 5줄: 힙 확장 실패면 에러 처리합니다.
- 8줄: 현재 starter는 block 맨 앞에 요청 크기만 기록합니다.
- 9줄: 사용자에게는 header 다음 주소, 즉 payload 시작 주소를 반환합니다.
- 핵심: `p`는 block 시작점이고, 반환값은 `p + header 크기`입니다.

starter 코드에서 지금 꼭 봐야 할 포인터

`p`는 allocator가 block 시작점을 관리하기 위한 포인터이고, `(char *)p + SIZE_T_SIZE`는 사용자가 쓰게 될 payload 시작 포인터입니다. 여기서 이미 **관리용 주소와 사용자용 주소가 다르다**는 감각을 잡을 수 있습니다.

5. 이번 주에 구현할 mm 함수들의 큰 흐름

함수	역할	재혁님이 포인터로 볼 포인트
mm_init	초기 힙 구조 생성	prologue, epilogue, heap_listp가 어디를 가리키는지
mm_malloc	맞는 free block을 찾아 배치	payload 크기와 block 전체 크기를 구분하는지
mm_free	block을 free 상태로 바꾸기	payload 포인터에서 header/footer를 역으로 찾는지
mm_realloc	크기 조정	옛 payload와 새 payload 사이 복사 범위를 이해하는지
extend_heap	힙을 더 늘려 free block 만들기	새 free block과 epilogue 재배치
find_fit	free block 탐색	next block으로 포인터가 어떻게 움직이는지
place	찾은 free block에 요청 block 배치	split 전후 block 경계가 어떻게 바뀌는지
coalesce	인접 free block 합치기	prev/next block을 어떻게 찾아 결합하는지

6. implicit free list에서 포인터가 실제로 어떻게 움직이나

implicit free list는 free block에 next 포인터를 저장하지 않습니다. 대신 **현재 block의 크기**를 읽고 그만큼 이동해서 다음 block으로 갑니다.

예시 3: block 순회 감각

```
1  void *bp = heap_listp;
2  while (GET_SIZE(HDRP(bp)) > 0) {
3      if (!GET_ALLOC(HDRP(bp))) {
4          return bp;
5      }
6      bp = NEXT_BLKP(bp);
7  }
8  return NULL;
```

- 1줄: 현재 보고 있는 block의 payload 포인터를 bp에 둡니다.
- 2줄: header에서 size를 읽고, size가 0인 epilogue 전까지 반복합니다.
- 3줄: alloc bit가 0이면 이 block은 free입니다.
- 4줄: 요청에 맞는 block을 찾으면 payload 포인터를 반환합니다.
- 6줄: 현재 block 크기만큼 더해서 다음 block의 payload 포인터로 이동합니다.
- 핵심: 여기엔 next 포인터가 없고, size를 이용한 주소 계산만 있습니다.

이해 포인트

재혁님이 지금까지 자주 헷갈린 부분이 바로 여기입니다. **implicit free list의 포인터는 "free block끼리 직접 연결된 포인터"가 아니라, 힙 전체를 순회하는 현재 위치 포인터**입니다.

7. mm_init: 왜 prologue와 epilogue를 만드는가

mm_init는 단순히 힙을 비우는 게 아니라, 나중에 coalescing과 순회를 쉽게 만들기 위해 "가짜 앞 블록"과 "가짜 끝 블록"을 세팅하는 함수라고 생각하면 좋습니다.

예시 4: mm_init에서 보고 싶은 코드 골격

```
1  int mm_init(void)
2  {
3      if ((heap_listp = mem_sbrk(4 * WSIZE)) == (void *)-1)
4          return -1;
5      PUT(heap_listp, 0);
6      PUT(heap_listp + (1 * WSIZE), PACK(DSIZE, 1));
7      PUT(heap_listp + (2 * WSIZE), PACK(DSIZE, 1));
8      PUT(heap_listp + (3 * WSIZE), PACK(0, 1));
9      heap_listp += (2 * WSIZE);
10     return 0;
11 }
```

- 3줄: 힙 시작 부분에 필요한 기본 공간을 확보합니다.
- 5줄: 맨 앞 padding word를 둬서 payload 정렬을 맞춥니다.
- 6줄: prologue header를 allocated 상태로 둡니다.
- 7줄: prologue footer도 같이 둡니다.
- 8줄: epilogue header를 size 0, allocated 1로 둡니다.
- 9줄: heap_listp를 prologue payload 위치 느낌으로 맞춥니다.
- 핵심: 경계 조건을 줄이기 위한 "가짜 블록" 설치 작업입니다.

8. mm_malloc: 요청 크기를 block 크기로 바꾸는 사고

사용자 관점

`malloc(15)`면 15바이트 쓰고 싶다는 뜻입니다.

allocator 관점

header/footer/정렬을 포함한 **block 전체 크기**를 계산해야 합니다.

그래서 `mm_malloc`는 보통 다음 순서로 생각합니다.

1. payload 요청 크기를 받음
2. header/footer overhead를 더함
3. 정렬 단위에 맞게 올림
4. free block에서 맞는 block을 찾음
5. 없으면 heap을 늘림
6. block을 배치하고 필요하면 split
7. payload 시작 주소를 반환

9. mm_free: free는 payload 포인터 하나만 받아서 어떻게 header를 찾나

`mm_free(void *ptr)`는 payload 포인터만 받습니다. 그런데 allocator는 header를 바꿔야 하죠. 그래서 **payload 포인터에서 거꾸로 4바이트 이동해서 header를 찾는 계산**이 필요합니다.

예시 5: free가 header/footer를 다시 쓰는 감각

```
1  void mm_free(void *bp)
2  {
3      size_t size = GET_SIZE(HDRP(bp));
4      PUT(HDRP(bp), PACK(size, 0));
5      PUT(FTRP(bp), PACK(size, 0));
6      coalesce(bp);
7  }
```

- 1줄: `bp`는 payload 시작 주소입니다.
- 3줄: `HDRP(bp)`로 header 위치를 찾고 size를 읽습니다.
- 4줄: header를 free 상태로 바꿉니다.
- 5줄: footer도 같은 정보로 갱신합니다.
- 6줄: 양옆 free block과 합칠 수 있는지 검사합니다.
- 핵심: free는 payload 포인터를 받아도, 거꾸로 header/footer를 찾을 수 있어야 합니다.

10. coalesce: 왜 포인터 공부의 핵심인가

`coalesce`는 포인터를 가장 잘 이해해야 하는 함수입니다. 이전 block, 현재 block, 다음 block의 상태를 보고 4가지 경우를 나눠야 하기 때문입니다.

경우	이전	다음	동작
Case 1	allocated	allocated	현재 block만 free로 둠
Case 2	allocated	free	현재 + 다음 합침
Case 3	free	allocated	이전 + 현재 합침
Case 4	free	free	이전 + 현재 + 다음 모두 합침

왜 footer가 필요한가

다음 block은 현재 block 크기를 알면 쉽게 찾지만, 이전 block은 바로 찾기 어렵습니다. 그래서 이전 free block의 footer를 읽어 이전 block 크기를 알고, 그 시작 주소를 역으로 계산합니다. 이게 boundary tag의 핵심입니다.

11. mm_realloc: 포인터 복사 감각 익히기

`realloc`은 포인터를 단순히 "새로 할당"만 보는 게 아니라, **기존 payload에서 새 payload로 몇 바이트를 복사해야 하는지**까지 생각해야 합니다.

예시 6: starter realloc 핵심 부분

```
1  void *mm_realloc(void *ptr, size_t size)
2  {
3      void *newptr = mm_malloc(size);
4      size_t copySize = *(size_t *)((char *)ptr - SIZE_T_SIZE);
5      if (size < copySize)
6          copySize = size;
7      memcpy(newptr, ptr, copySize);
8      mm_free(ptr);
9      return newptr;
10 }
```

- 3줄: 먼저 새 payload 공간을 확보합니다.
- 4줄: 옛 payload 포인터에서 뒤로 가서 기록된 크기를 읽습니다.
- 5~6줄: 새 요청 크기보다 많이 복사하면 안 되니 최소값을 택합니다.
- 7줄: payload 데이터만 새 위치로 복사합니다.
- 8줄: 옛 block을 free합니다.
- 핵심: `realloc`은 "옮기기"보다 "새로 만들고 복사하고 정리하기"로 먼저 이해해도 좋습니다.

12. 앞으로 구현할 helper 함수들까지 한 번에 보기

helper	한 문장 설명	포인터 핵심
`extend_heap`	힙 끝을 늘려 큰 free block을 하나 추가한다	새 epilogue 위치를 다시 잡아야 함
`find_fit`	implicit free list를 순회하며 맞는 free block을 찾는다	`bp = NEXT_BLKP(bp)` 감각이 중요
`place`	찾은 free block을 할당하고 남는 조각은 split한다	앞 block과 뒤 remainder block 경계를 다시 씀
`coalesce`	인접 free block들을 합쳐 큰 free block으로 만든다	prev/next block 주소 계산이 가장 중요

13. 수요 코딩회(B+트리)와 포인터 연결

재혁님이 이번 수요 코딩회에서 B+트리를 구현할 때도 포인터 감각은 그대로 이어집니다. 단지 block 대신 **node**를 다룰 뿐입니다.

예시 7: 트리 노드 포인터 감각

```
1  typedef struct BNode {
2      int is_leaf;
3      int key_count;
4      int keys[ORDER - 1];
5      struct BNode *children[ORDER];
6      struct BNode *next;
7  } BNode;
```

- 1줄: 트리 노드 구조체를 정의합니다.
- 2줄: 현재 노드가 리프인지 내부 노드인지 표시합니다.
- 3줄: 현재 몇 개의 키를 들고 있는지 저장합니다.
- 4줄: 키 배열입니다.
- 5줄: 자식 노드 주소들을 저장합니다.
- 6줄: B+트리 리프 노드끼리 옆으로 연결할 때 쓰는 포인터입니다.
- 핵심: malloc-lab의 block 포인터가 힙 블록을 가리켰다면, 여기선 노드 포인터가 트리 구조를 가리킵니다.

malloc-lab과 수요 코딩회의 공통점

- 둘 다 포인터가 "데이터"보다 "구조"를 잇는 역할을 합니다.
- malloc-lab은 block과 block의 경계를 계산하는 포인터를 다루고,
- B+트리는 node와 child를 잇는 포인터를 다룹니다.
- 둘 다 **내가 지금 어떤 주소를 가리키는지**를 잃어버리면 바로 헷갈립니다.

14. 오늘 재혁님이 이렇게 공부하면 좋습니다

1. 이 문서의 **2~4장**을 읽고, payload/header/footer가 각각 어디인지 소리 내어 말해보기
2. **6장**을 보면서 `bp`가 다음 block으로 어떻게 이동하는지 손으로 주소 계산해보기
3. **9장과 10장**을 보고, free와 coalesce가 왜 payload 포인터만으로도 가능한지 설명해보기
4. **12장**을 보고 이번 주 구현해야 할 함수들의 역할을 한 줄씩 직접 적어보기
5. **13장**을 보고 수요 코딩회에서 포인터가 트리 구조로 어떻게 바뀌는지 연결해보기

15. 오늘 끝나기 전에 스스로 답해볼 8문장

1. `malloc`이 반환하는 건 왜 header 주소가 아니라 payload 주소인가?
2. payload 크기와 block 전체 크기는 왜 다를 수 있는가?
3. implicit free list에서 "다음 block"은 무엇을 보고 찾는가?
4. `mm_free`는 payload 포인터만 받아서 어떻게 header를 다시 찾는가?
5. coalesce에서 이전 block을 찾기 어려운 이유는 무엇인가?
6. boundary tag는 왜 footer를 도입하는가?
7. `extend_heap`은 왜 epilogue를 다시 써야 하는가?
8. B+트리에서 포인터는 block이 아니라 무엇을 가리키는가?

오늘의 목표를 너무 크게 잡지 않기

오늘은 `mm.c`를 완성하는 날이 아니라, 내일부터 구현할 때 안 무너지도록 **포인터의 역할을 눈으로 구분하는 날**입니다. 이 문서를 다 보고 나서 위 8문장 중 6문장 이상을 자기 말로 설명할 수 있으면 아주 잘한 겁니다.

마지막 한 줄 정리

포인터는 값을 읽기 위한 도구이기도 하지만, 이번 주에는 "메모리 구조를 따라다니는 지도"라고 생각하면 가장 이해가 잘 됩니다.